Deep Learning, Assignment 2

Maksym Pasko, Artem Korniienko

Code: github.com/akorniienko04/Temych_and_Masik_CNN W&B project: `akorniienko-kse/cats-dogs-pp`

Contents

1 Task and data

2 Data insights

3 Metric analysis

4 Validation

4.1 Train/test split analysis

4.2 Choosing a scheme

5 CNN model

5.1 Setup

5.2 Experiments

5.3 Own block: MLP head

5.4 Health checks

5.5 Is the model adequate?

6 Ensemble

7 Results on Kaggle

8 Final hyperparameters

9 Ideas that didn't work

10 Limitations

11 Conclusions

A AI usage

1 Task and data

The task is to find which of five labels are present in a photo: `Cat`, `Dog` and three specific pets, `Moris`, `Motya` and `Biatrix`. It is **multi-label**: one photo can have several labels or none. The metric is **F1 macro**. The data comes from the Kaggle In-Class competition `cats-and-dogs-plus-plus`: 12,260 train photos and 18,390 test photos with hidden labels. Photos have different sizes (e.g. 375 × 450).

The code is in four notebooks, each one runs on its own (Table 1).

Table 1: Notebooks.

Notebook	Description
`CNN_eda`	data exploration and metric
`CNN_val`	validation scheme
`CNN_model_clean`	CNN experiments, gradient flow, GradCAM
`CNN_ensemble`	ensemble

2 Data insights

- **Strong imbalance.** Cats and dogs have about 5,000 photos each, the three pets only 41–56 each, about 0.4% of train (Table 2). In F1 macro every label has the same weight, so the three rare pets are 60% of the score.
- **Really multi-label.** 2,162 photos (17.6%) have no label at all, 125 photos have two or more. So the model needs 5 independent yes/no outputs (sigmoid), not one softmax that always picks a class.
- **No missing values** in the label table.

Table 2: Labels in train.

Label	Photos	Share
Cat	5,077	41.4%
Dog	5,025	41.0%
Moris	56	0.46%
Motya	42	0.34%
Biatrix	41	0.33%
no label	2,162	17.6%
2+ labels	125	1.0%

3 Metric analysis

What F1 macro is. For each label we compute $F1 = 2PR/(P + R)$, where precision P is the share of correct "yes" predictions and recall R is the share of true "yes" that we found. F1 macro is the average over the 5 labels.

Strengths. Accuracy is useless here (always saying "no" for the pets is already 99.6% correct for them). F1 ignores easy true negatives and needs both precision and recall. Macro averaging forces the model to care about the rare pets.

Weaknesses and edge cases.

- It needs a threshold. We use 0.5, which is not necessarily best for rare labels.
- It ignores confidence: probabilities 0.51 and 0.99 count the same.
- A correct "no animal" on the 2,162 unlabeled photos gives no credit.
- If the model never predicts a rare pet, its F1 is 0 and the score loses 0.2.
- It is very noisy: a validation fold has only 8–11 photos of each pet, so one mistake changes that pet's recall by about 10%.
- A trivial "all ones" submission scores ≈ 0.24.

Other metrics. We also look at F1, precision and recall per label, and micro F1.

Alternative target metric. Macro PR-AUC (average precision per label, then averaged). It keeps the equal weight of labels and focuses on the positive class like F1, but it does not need a threshold and uses the model's confidence.

Conclusion. We keep F1 macro as the main metric, since the competition is scored on it, and look at per-label F1 to see where the model fails.

4 Validation

4.1 Train/test split analysis

Time. We checked 500 train and 500 test photos: none has EXIF metadata (no date, no camera model), and file names are hashes. So a time factor cannot be checked, and a time split is not possible.

Groups. We looked for copies and photo series:

- MD5 of the files: 5 pairs of exact duplicates in train, all with the same labels (no labeling errors), and 14 exact duplicates between train and test, negligible for 18,390 test photos.
- Perceptual hash (64 bits that describe how a photo looks) and the Hamming distance between hashes. There is a clear gap: 162 pairs at distance 0–2 (practically the same photo), almost nothing at 3–9, and hundreds of pairs from 10 on, which are already different photos (Figure 1).

162 pairs touch at most 324 photos, about 2.6% of train. So series can leak between train and validation only for a small part of the data.



Figure 1: Pairs at Hamming distance 0: the same photo twice. Some photos show other animals (otters, birds, goats).

4.2 Choosing a scheme

We considered two schemes:

- `MultilabelStratifiedKFold`: every fold gets the same share of each label. Without it a fold could get no Motya at all.
- `StratifiedGroupKFold`: the same, plus all photos of one series stay in one fold. Its only advantage is protection from series leakage.

Series cover only about 2.6% of train, so this advantage is small, while stratification of the rare pets is critical. We chose `MultilabelStratifiedKFold`, 5 folds, `random_state=42`. The folds are saved to `folds.csv` and reused in every experiment, so all runs are compared on the same split (Table 3).

Table 3: Labels per fold.

Fold	Cat	Dog	Moris	Motya	Biatrix
0	1015	1005	11	8	9
1	1016	1005	12	8	8
2	1015	1005	11	9	8
3	1016	1005	11	8	8
4	1015	1005	11	9	8

Because of the compute limits of Colab, all experiments train on folds 1–4 and validate on **fold 0** (9,808 / 2,452 photos).

Adversarial validation. We did not run it: CV and the leaderboard pick the same best single models (run5, then run4; Section 7), so there is no big mismatch that would call for it.

5 CNN model

5.1 Setup

- **Framework.** PyTorch Lightning for the training loop, W&B for logs, checkpoints saved to Google Drive.
- **Architecture.** EfficientNet-B0 from `timm` (`efficientnet_b0.ra_in1k`), pretrained on ImageNet-1k only, as the rules require. It is small (4.0M parameters with our 5-label head), fast enough for Colab and strong for its size.
- **Input.** Resize to 224 × 224, ImageNet mean/std normalization (the same as during pretraining). Train augmentation: random horizontal flip. No augmentation for validation and test.
- **Loss and output.** 5 logits, `BCEWithLogitsLoss` (one binary loss per label), sigmoid, threshold 0.5.
- **Training.** Batch 32, 5 epochs, mixed precision, best epoch by validation F1 macro.

5.2 Experiments

We change one thing at a time (Table 4).

Table 4: Experiments, validation on fold 0.

Run	Change	Optimizer	LR	Scheduler	Head	CV F1	Public LB
run1	baseline	Adam	1e-3	–	linear	0.767	0.767
run2	optimizer	AdamW	1e-3	–	linear	0.830	0.758
run3	scheduler	AdamW	1e-3	plateau	linear	0.829	0.852
run4	learning rate	AdamW	3e-4	plateau	linear	0.908	0.884
run5	own block	AdamW	3e-4	plateau	MLP	**0.945**	**0.918**

- **AdamW vs Adam.** Better on CV (0.830 vs 0.767), but not on the leaderboard (0.758 vs 0.767), so this gain is not confirmed.
- **ReduceLROnPlateau** (halve the LR when validation F1 stops growing, patience 1) never fired within 5 epochs: the LR stays flat in Figure 2. So run3 is in fact the same training as run2. Their CV is the

same (0.829 vs 0.830), but the leaderboard differs by 0.094, which shows how noisy the scores are with so few rare pets.

- **Lower learning rate** gave the biggest gain: 3e-4 instead of 1e-3, +0.079 on CV and +0.032 on the leaderboard. 1e-3 is too high for fine-tuning: it destroys the pretrained features.

5.3 Own block: MLP head

The default head is one linear layer 1280 → 5. We replace it with our own block:

$$\text{Linear}(1280 \to 512) \to \text{BatchNorm} \to \text{ReLU} \to \text{Dropout}(0.3) \to \text{Linear}(512 \to 5)$$

(659K parameters). Hypothesis: an extra non-linear layer can combine features better for the rare pets, and Dropout reduces overfitting. All other settings are the same as in run4.

Result. CV 0.945 vs 0.908, leaderboard 0.918 vs 0.884. Validation loss is lower and train loss slightly higher than with the linear head (Figure 2), so Dropout reduced overfitting. A repeated run of the same config gave 0.893, so part of the CV gain is noise, but the leaderboard confirms the improvement.



Figure 2: W&B curves for run4 (linear head) and run5 (MLP head): validation loss and F1, train loss, learning rate.

5.4 Health checks

Curves. Validation F1 peaks at epoch 3 for run5 and at epochs 2–3 for run4, then run4 drops to about 0.86 (Figure 2). Checkpointing on the best epoch protects from this.

Gradient flow. We computed gradients on 5 batches without updating the weights and averaged |grad| per layer. Gradients reach every block, including the first one (`conv_stem`); values range from about $6 \cdot 10^{-5}$

(`conv_head`) to $4 \cdot 10^{-3}$ (`conv_stem`), with no zeros and no spikes (Figure 3). So there is no vanishing or exploding gradient.

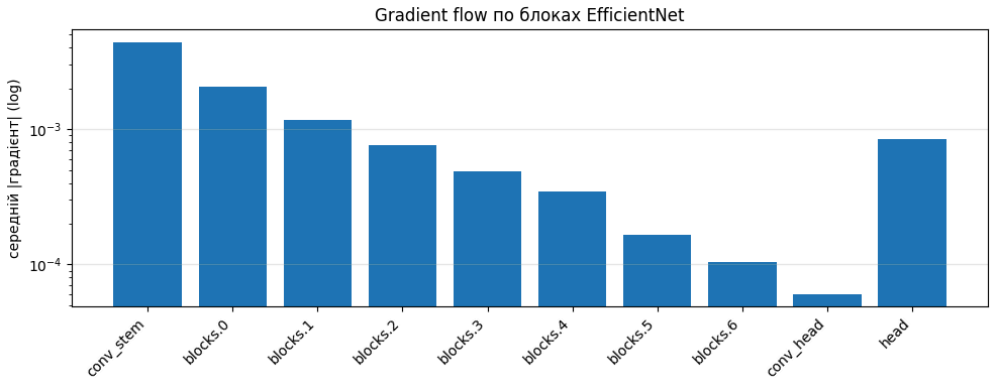


Figure 3: Mean absolute gradient per EfficientNet block, run5 (log scale).

GradCAM. On the last convolutional layer (`conv_head`), for the first validation photo of each label (Figure 4). For Cat, Dog, Moris and Biatrix the model looks at the animal (face or body). For Motya it looks at the top of the head and partly at the background above it, so it may also use the context.

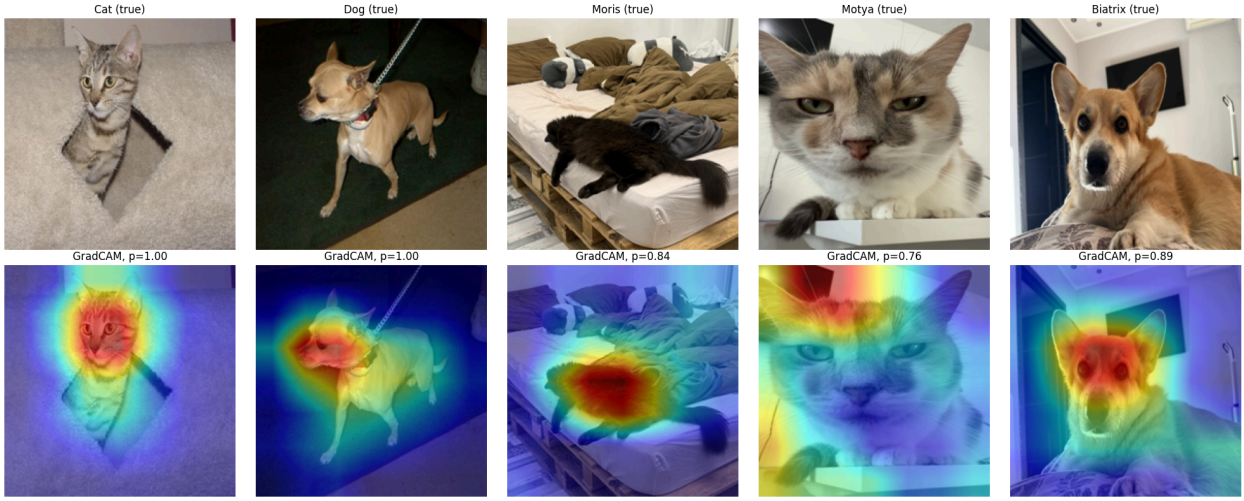


Figure 4: GradCAM for one validation photo per label, with the predicted probability.

5.5 Is the model adequate?

Yes. F1 grew from 0.767 to 0.945 on CV and from 0.767 to 0.918 on the leaderboard, CV and the leaderboard pick the same best model, training is healthy, and GradCAM shows that the model looks at the animals, including the rare pets.

6 Ensemble

Method. Soft voting: we average the predicted probabilities of several CNNs and apply the 0.5 threshold. There is no meta-model, so this is not stacking.

Candidates. run3 and run4 (linear head) and run5 (MLP head). We compare two ensembles, run4 + run5 and run3 + run4 + run5, and pick the one with the best CV on fold 0, not by the leaderboard (Table 5).

Table 5: Single models and ensembles, CV on fold 0.

Model	CV F1	Public LB
run4 (linear head)	0.908	0.884
run5 (MLP head)	0.945	**0.918**
ensemble run4 + run5	**0.948**	0.910
ensemble run3 + run4 + run5	0.936	–

We chose run4 + run5 by CV (0.948). It is only slightly above run5 alone (0.945), within the run-to-run noise, and on the leaderboard it is worse (0.910 vs 0.918): run4 is clearly weaker than run5, and averaging pulls the predictions towards it. So the ensemble does not improve the result, and run5 stays our best model. Adding run3 lowers CV even more (0.936), and this ensemble was not submitted. Per-label F1 of the chosen ensemble: Cat 0.989, Dog 0.986, Moris 1.000, Motya 0.824, Biatrix 0.941; Motya is the weakest label, which matches the GradCAM observation above.

7 Results on Kaggle

Table 6: CV vs Public leaderboard for all submissions.

Model	CV F1 (fold 0)	Public LB
run1, baseline	0.767	0.767
run2, AdamW	0.830	0.758
run3, + plateau	0.829	0.852
run4, LR 3e-4	0.908	0.884
run5, MLP head	0.945	0.918
ensemble run4 + run5	0.948	0.910

- Among single models, the best model is the same on CV and the leaderboard (run5, then run4), and large gains show up on both. The rank correlation over the 5 single runs is 0.7.
- CV is 0.02–0.03 higher than the leaderboard, so our validation is slightly optimistic. Likely reasons: we pick the best epoch on the same fold we report, and fold 0 has only 8–11 photos of each pet.
- Small differences are not reliable on either side: run2 vs run3, and the ensemble vs run5 (+0.003 on CV, −0.008 on the leaderboard).

8 Final hyperparameters

Table 7: Hyperparameters of the final model (run5).

Parameter	Value
Backbone	EfficientNet-B0, ImageNet-1k (`ra_in1k`)
Head	Linear 1280→512, BatchNorm, ReLU, Dropout 0.3, Linear 512→5
Input	224 × 224, ImageNet normalization, horizontal flip
Loss	BCEWithLogitsLoss
Optimizer	AdamW, LR 3e-4
Scheduler	ReduceLROnPlateau (factor 0.5, patience 1; never fired)
Training	batch 32, 5 epochs, mixed precision, best epoch by val F1 (epoch 3)
Threshold	0.5 for all labels

9 Ideas that didn't work

Table 8: Ideas that didn't work.

Idea	Result	Why
ReduceLROnPlateau	never reduced the LR	with patience 1, F1 kept growing a little within 5 epochs
AdamW instead of Adam	CV +0.063, LB −0.009	the CV gain was mostly noise of the rare pets
LR 1e-3 for fine-tuning	0.767–0.830 CV vs 0.908 at 3e-4	a high LR destroys the pretrained features
StratifiedGroupKFold	not used	series cover only ~2.6% of train, stratification matters more
Ensemble run4 + run5	CV +0.003, LB −0.008 vs run5	run4 is weaker than run5; the CV gain was within noise

10 Limitations

- All experiments use only fold 0. With 8–11 photos of each pet in it, run-to-run noise (0.945 vs 0.893 for the same config) is as big as most differences between runs.
- The best epoch and the best checkpoint are chosen on the same fold we report, which makes CV optimistic.
- The threshold is 0.5 for every label and was not tuned for the rare pets.
- Resizing to 224 × 224 does not keep the aspect ratio.

11 Conclusions

We explored the data, analysed the F1 macro metric, checked the split for time and group factors, built a stratified 5-fold scheme, fine-tuned EfficientNet-B0 with PyTorch Lightning and W&B, added our own

MLP head, checked the training with curves, gradient flow and GradCAM, and combined several CNNs with soft voting.

What worked well.

- A lower learning rate for fine-tuning gave the biggest gain (+0.079 CV).
- The MLP head improved both CV and the leaderboard (0.918 public), and it is our final model.
- CV and the leaderboard agree on the best single model, so the validation scheme works.

What worked less well.

- The scheduler never fired, and the AdamW gain was not confirmed on the leaderboard.
- The ensemble did not beat run5 alone on the leaderboard (0.910 vs 0.918).
- With so few rare pets, single-fold scores are noisy.

Room for improvement. Train on all 5 folds and average them, tune the threshold per label, use stronger augmentations and a larger input size for the rare pets.

A AI usage

We used Claude (Anthropic, Claude Code desktop app) as an assistant during the assignment:

- **Explanations of concepts:** convolution, pooling, BatchNorm and LayerNorm, receptive field, LoG and impulse response, inductive bias, transfer learning, F1 macro, validation schemes.
- **Planning:** splitting the assignment into steps and checking which requirements were covered.
- **Code assistance for specific parts:** data loading with `kagglehub`, `MultilabelStratifiedKFold` folds, EXIF and duplicate checks (MD5, perceptual hash), Dataset and DataLoaders, the LightningModule with W&B logging, experiment and submission helpers, the MLP head, gradient flow and GradCAM plots, the ensemble notebook, notebook cleanup.
- **Language:** grammar correction of our texts and help with the report wording and layout.

All code was run, debugged and checked by us in Colab. The full chat transcript is attached to the submission.